

CYBR 437 Secure Coding

Winter 2026

Lab - 6

Release date: February 10, 2026

Due date: February 17, 2026, at 11:59 pm

Introduction

In this lab, you will gain hands-on experience analyzing binaries and exploiting buffer overflow vulnerabilities with return-oriented programming. You will execute the following buffer overflow attacks to gain insight into the stack's operation and how insecure code can lead to control flow manipulation.

Setup

- A lab6 tarball is available for download from the Lab-6 assignment page, within are 2 binary files `file1` and `file2`.
- Upload the tarball to your VM and untar with the following commands:
 - `scp ./lab6.tar.gz [username]@[ip]:./[directory]`
 - `tar -xzvf lab6.tar.gz`
 - Make the binaries executable with `chmod +x <filename>`
- Ensure that ASLR is turned off using the following or other commands:
 - `echo 0 | sudo tee /proc/sys/kernel/randomize_va_space` (to turn off)
 - `cat /proc/sys/kernel/randomize_va_space` (output of 0 indicates it is off)

Tasks (25 points per task)

Task 1

Analyze the binary `file1` to find the user-defined functions and how (or whether) they are called. You will observe that only one of the user-defined functions is called in main that takes a string as input (which causes a vulnerability). Exploit this vulnerability so that the program does not return to main after calling that function, and instead calls the same function again, and terminates without a crash, such as a segmentation fault.

Task 2

Using the same file as in Task 1, exploit the vulnerability to call the other user-defined function, which prints a flag, similar to the flags in Lab-5. After revealing the flag, the program should terminate without crashing.

Task 3

Analyze the binary `file2` to find the user-defined functions and how (or whether) they are called. You will observe that only one of the user-defined functions is called in main. This program takes a command-line string as input, which causes a vulnerability. Exploit this vulnerability to call the

other two user-defined functions to reveal two flags (each of the functions prints one). The program should finally terminate without crashing.

Submission Instructions

Your write-up submission should include:

- For every problem, screen captures demonstrating:
 - Checking the binary file's endianness with the `file` command.
 - Checking which defenses have been turned on/off with `checksec`.
 - The use of `pwndbg` to illustrate how you arrived at the value or the eip.
 - Your success in each task (screenshots should include the entire output)
- Detailed explanations of how you performed each buffer overflow attack, including tools and techniques used.